Data-Driven Discovery of Interpretable Kalman Filter Variants through Large Language Models and Genetic Programming: Supplementary Material

Vasileios Saketos[1,2][0009−0000−8205−4528], Sebastian Kaltenbach[1][0000−0002−4261−7282], Sergey Litvinov[1][0000−0001−9152−4835], and Petros Koumoutsakos[1][0000−0001−8337−2122]

[1] Computational Science and Engineering Laboratory, Harvard University, Cambridge, USA
[2] KTH Stockholm, Sweden
petros@seas.harvard.edu

1 Binpacking Test Case

We verified our LLM-assisted ES implementation by applying it to the bin-packing task as defined in the DeepMind FunSearch paper [3]. By doing so, we are able to benchmark our implementation against FunSearch. We note that apart from potential differences in the implementation, we also rely on a LLM from DeepSeek [2] in contrast to a Gemini variant for FunSearch.

The experiments are based on standard benchmark datasets from the OR-Library [1], including `binpack1`, `binpack2`, `binpack3`, and `binpack4`. Each dataset contains 20 bin packing instances with 120, 250, 500, and 1000 items, respectively. The sizes of the items are sampled uniformly from the interval $[20, 100]$ and the bin capacity is fixed at 150. Following the original setup, we generated a training dataset of 20 instances with 120 items (mirroring `binpack1`) and a validation dataset of 20 instances with 250 items (similar to `binpack2`). Candidate heuristics were evolved using our LLM-assisted ES and selected based on validation performance. The best performing programs were then evaluated on the complete benchmark suite (`binpack1`–`binpack4`) to assess generalization. This reproduction served to confirm that our implementation faithfully replicates the original methodology and yields consistent performance behavior.

In Figure 1 and Table 1, we compare the performance of three heuristics on the OR-Library bin packing benchmarks: the standard Best Fit heuristic, the heuristic discovered by the original DeepMind FunSearch framework, and the one produced by our own implementation.

Across all datasets, both Deepmind FunSearch and our implementation outperform the default Best Fit heuristic, confirming that learned algorithms lead to more efficient bin usage. In `binpack1.txt`, both Deepmind FunSearch and our algorithm achieve a 5.30% excess over the L1 bound, improving on the Best Fit heuristic's 5.81%. In `binpack2.txt`, Deepmind FunSearch achieves

the best result (4.19%), with our method slightly behind at 4.92%, both improving significantly over Best Fit heuristic's 6.06%. A similar trend holds for `binpack3.txt`, where Deepmind FunSearch yields the lowest excess (3.11%), followed by our result at 4.20%, again outperforming the Best Fit heuristic (5.37%). In `binpack4.txt`, the pattern continues: Deepmind FunSearch produces the best excess (2.47%), our method is second (3.92%), and Best Fit trails behind (4.94%). Regarding computational cost, our method was run for six days on four H100 GPUs. The original DeepMind FunSearch paper reports the heuristic used for bin packing, but does not disclose the exact number of runs required to obtain it. Based on the scale of their capset experiments (which involved 140 separate runs, each using 15 A100 GPUs for two days), it is possible that their original search could have involved substantially more compute than our replication. Despite this, our results remain highly competitive.
These results demonstrate that our implementation is capable of discovering heuristics that outperform the current Best Fit heuristic and are competitive with those produced by DeepMind's FunSearch framework.

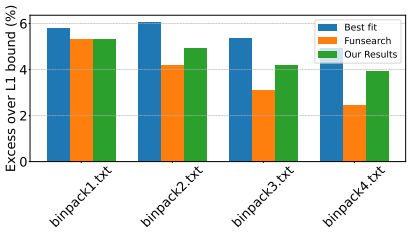

Fig. 1: Comparisons of the accuracy of discovered algorithms for the bin packing task.

Dataset	FunSearch	Our Results	Best Fit
`binpack1`	5.30%	5.30%	5.81%
`binpack2`	4.19%	4.92%	6.06%
`binpack3`	3.11%	4.20%	5.37%
`binpack4`	2.47%	3.92%	4.94%

Table 1: Excess over L1 bound (%) for each method on binpack datasets. Lower is better. Colors indicate performance per row: green = best, yellow = middle, red = worst.

2 Discovered algorithms

In Figure 2, we present the algorithms discovered by FunSearch and CGP for the delayed observation scenario, while Figure 3 shows the corresponding solutions for the case of nonlinear state transitions. These examples highlight the structural differences between the two approaches. LLM-assisted ES tends to produce programs with greater algorithmic richness and mathematical depth, making use of a wide range of operations, including matrix products, element-wise nonlinearities, and adaptive scaling mechanisms. In contrast, the CGP-generated functions are more constrained in form and typically rely on a pre-defined, fixed-size set of operations.

One of the key strengths of LLM-assisted ES is its ability to discover expressive and complex update rules that go beyond the structure of classical filtering techniques. For instance, the inclusion of operations such as `np.log`, `np.tanh`, and sinusoidal terms in LLM-assistes ES outputs reflects an inherent flexibility to model richer dynamics or nonlinear observation processes. Moreover, in our case, these algorithms often maintain recognizable algorithmic blocks — such as state predictions x_p, covariance updates P, and innovation terms y — which can be directly associated with standard Kalman filter notation. This facilitates theoretical analysis and easily shows in which part the newly discovered algorithm differs from the established baseline.

```python
def function_approximate(x, F, P, Q,
    z, R):
    a =F @x
    b =F @np.log(np.maximum(a *0.03,
        1e-8))
    c =F @np.tanh(b *0.2)
    xp =a +c
    P =F @P @F.T +0.7 *Q
    y =z -xp
    S =P +0.7 *R
    inv_S =np.linalg.inv(S)
    K =P @inv_S +0.15 *inv_S
    x =xp +K @y
    x +=0.6 *F @np.tanh(F @x *0.08)
    P =(np.eye(F.shape[0]) -K) @P
    return xp, P, y, S, K, x
```

```python
def graph_approximate(x, F, P, Q, z, R
    ):
    A =F @R
    B =P +Q
    K =np.linalg.inv(B)
    S =Q @K
    y =A -x
    T =K.T
    P =S -S
    F_ =S +F
    xp =y @T +x
    x =y @T +x
    S =F_ @S
    K =T -P
    return xp, P, y, S, K, x
```

Fig. 2: Algorithms discovered for delayed observations by LLM-assisted ES on the left and CGP on the right.

```python
def function_approximate(x, F, P, Q,
    z, R):
    x =np.array(
        [0.04 *x[0]**3 -1.8 *x[0] +0.3
            4 *np.sin(x[1]),
                0.14 *np.tanh(0.05 *
                    x[0] *(x[1] +0.8
                    ))])
    xp =F.dot(x)
    scale_Q =(x[0] *x[1] +x[0]**2 +x[
        1]**2 +0.6) *\
                (1 +0.9 *(x[0] *x[1] +
                    x[0]**2 +x[1]**2)
                    )
    P =F.dot(P.dot(F.T)) +Q *scale_Q
    y =z -xp
    scale_R =(x[0]**2 +x[1]**2 +0.5)
        *(1 +0.8 *(x[0]**2 +x[1]**2))
    S =P +R *scale_R
    inv_S =np.linalg.inv(S +0.0002 *
        np.eye(S.shape[0]))
    K =P.dot(inv_S) *(0.85 *np.tanh(
        np.linalg.norm(y)))
    x =xp +K.dot(y)
    P =(np.eye(F.shape[0]) -K) *P *(0
        .5 +0.05 *np.mean(y**2))
    return xp, P, y, S, K, x
```

```python
def graph_approximate(x, F, P, Q, z, R
    ):
    xp =x
    x =np.array([0.05 *x[0]**3 -2*x[0
        ], 0.1 *np.sin(x[1])])
    Ft =F.T
    y =z -(F @x)
    S =Ft.T +Ft +P +Q
    K =Ft @np.linalg.inv(S)
    P =K @y

    x =x +P
    return xp, P, y, S, K, x
```

Fig. 3: Algorithms discovered for non linear transitions by LLM-assisted ES on the left and CGP on the right.

3 Prompts for LLM-assisted ES

Figure 4 shows the prompt used to guide the language model during the search procedure. The model is instructed to generate multiple independent implementations of a single Python function with a fixed interface. While the input–output structure is strictly constrained, the internal computation is left deliberately open, encouraging the emergence of diverse algorithmic strategies. Additional stylistic restrictions, such as the exclusion of comments and the requirement that all outputs are NumPy arrays, are imposed to focus the search on functional behavior rather than presentation. This design allows to explore a broad space of candidate algorithms while maintaining comparability across generated solutions.

```
### Task:
# Generate 10 unique implementations of a Python function named aproximate.

# Requirements:
# 1. Function Signature:
# def aproximate(x, F, P, Q, z, R):
#
# 2. Output:
# Each function must return a tuple of six NumPy arrays.
#
# 3. Constraints:
# - Use NumPy operations creatively.
# - Do not include comments or markdown formatting inside the functions.
#
# 4. Additional Notes:
# - All returned outputs must be NumPy arrays with matching dimensions where
    appropriate.
# - You may introduce global variables in the function to track state or
    trajectory (there is a global list history for that reason), but keep
    the implementations minimal and elegant.
```

Fig. 4: Prompt provided to the language model in the LLM-assisted ES framework.

References

1. Beasley, J.E.: Or-library: Distributing test problems by electronic mail. Journal of the Operational Research Society **41**(11), 1069–1072 (1990)
2. Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025)
3. Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J.R., Ellenberg, J.S., Wang, P., Fawzi, O., Kohli, P., Fawzi, A.: Mathematical discoveries from program search with large language models. Nature **625**(7995), 468–475 (2024). https://doi.org/10.1038/s41586-023-06924-6, https://doi.org/10.1038/s41586-023-06924-6